PROJECT DETAIL • SOFTWARE SPECIFICATION

NutriSense

AI-Powered Nutrition, Diet & Clinical Health Companion

Technical Project Detail & Architecture Document

Prepared by **Jamal Matloob**

Version 1.0 | August 2026 | Confidential — Internal Use

Table of Contents

1. Executive Summary & Project Introduction

NutriSense is an enterprise-grade, AI-powered nutritional intelligence, fasting management, and clinical health platform. Engineered to combat the global surge in lifestyle-related metabolic disorders—principally Type 2 Diabetes, Hypertension, Cardiovascular Complications, and Obesity—NutriSense delivers a zero-friction dietary tracking experience grounded in medical guardrails, localized South Asian and global culinary intelligence, and multi-member family support.

1.1 Purpose & Problem Statement

Conventional dietary tracking applications impose severe cognitive friction on users by demanding manual numerical entry of grams, calories, and macronutrients. Furthermore, mainstream platforms suffer from critical deficiencies:

- Manual Data Entry Burnout: Requiring users to weigh portions and look up raw macros results in over 70% user abandonment within the first 14 days.
- Western Diet Bias: Existing nutritional databases offer inaccurate macro breakdowns for regional, South Asian, and Middle Eastern mixed dishes (e.g., Nihari, Haleem, Aloo Paratha, Daal, Biryani).
- Absence of Religious Fasting Support: Generic health apps fail to adjust nutritional goals, hydration pacing, and meal slots during Ramadan or intermittent fasting regimens.
- Isolated Single-User Silos: Modern families lack the tooling to manage elderly parents with hypertension or diabetic children from a consolidated, secure portal.

1.2 Target Audience

NutriSense is architected for four primary demographic cohorts:

1. Health-Conscious Individuals seeking friction-free, AI-assisted calorie and macronutrient balancing.

2. Patients Managing Chronic Conditions (Diabetics, Hypertensive, Hypercholesterolemia, PCOS) requiring strict ingredient safety cross-checks.

3. Multi-Member Households coordinating dietary preferences, pediatric allergies, and geriatric health targets.

4. Fasting Practitioners observing Ramadan or intermittent fasting protocols needing astronomical prayer-synchronized timers and hydration coaching.

1.3 Key Value Propositions & Core Differentiators

- Zero-Guesswork AI Meal Logger: Users simply type or state their meal in natural English or Urdu (e.g., '2 Aloo Parathas and 1 cup Chai'); Gemini AI parses portions and automatically computes complete macronutrient metrics.
- Multi-Key High-Availability Failover Pool: Dynamic round-robin API key rotation guarantees 99.9% uptime and zero rate-limiting (HTTP 429) across high-volume inference loads.
- Ramadan Fasting Intelligence: Location-aware astronomical solar calculations compute real-time Sehri/Iftar countdowns, fasting rings, and hydration pacing presets.
- Family Dependent Management: Single-account multi-profile tracking with isolated medical rules, custom calorie goals, and instant dependent dashboard switching.

• Emergency Clinic & Hospital Radar: Real-time geospatial queries discover nearby certified emergency clinics and hospitals with 1-tap turn-by-turn routing.

• 100% Offline Resilience: SQLite local caching guarantees full offline capability with transactional background synchronization to Supabase Cloud.

2. System Architecture & Tech Stack Specification

NutriSense adopts a modern, decoupled, multi-tier client-server architecture designed for high availability, sub-second latency, and enterprise security compliance.

2.1 High-Level Layered Architecture Overview

The system is organized into five distinct architectural tiers:

1. Presentation Layer (Client): Multi-platform Flutter application providing responsive, high-contrast dark-mode UI with bilingual Urdu/English localization.

2. State & Service Controller Layer: Reactive ListenableBuilder controllers, Singleton ViewModels, and background sync workers managing local business logic.

3. Backend API Gateway Layer: Asynchronous FastAPI microservices providing secure REST endpoints, input sanitization, and structured JSON schemas.

4. AI & Intelligence Layer: Google Gemini LLMs governed by a multi-key failover pool and clinical safety guardrail engines.

5. Data Persistence Layer: Supabase PostgreSQL with Row Level Security (RLS) policies coupled with an offline-first mobile SQLite caching layer.

2.2 Frontend Architecture & Client-Side Stack

The mobile and web client is engineered using Flutter 3.x, prioritizing native performance, modularity, and accessibility.

Component Layer	Technology / Library	Architectural Purpose & Scope
Framework	Flutter 3.x / Dart 3.x	Cross-platform high-performance rendering engine (Android, iOS, Web).
State Management	Provider & ListenableBuilder	Lightweight, reactive state propagation for ViewModels and controllers.
Local Database	sqflite / SQLite	Offline-first transactional local cache for meal logs, water logs, and habits.
Preferences	shared_preferences	Secure local persistence for tokens, language, and Ramadan state flags.
Health Integration	health package (v12+)	Bi-directional bridge to Google Health Connect and Apple HealthKit.
Voice & Speech	flutter_tts	Bilingual Text-To-Speech audio synthesizer for English and Urdu coaching.
Networking	http & connectivity_plus	REST API communication with dynamic network status monitoring.

2.3 Backend Architecture & Server-Side Stack

The backend infrastructure is built on FastAPI (Python 3.11+ ASGI), chosen for native asynchronous I/O and rapid JSON serialization.

Service Module	Core Technology	Operational Capability & Responsibilities
API Server	FastAPI / Uvicorn (ASGI)	Asynchronous endpoints handling meal search, coaching, and sync.
Cloud Database	Supabase PostgreSQL	Relational database with strict Row Level Security (RLS) and Auth.
Authentication	Supabase Auth (JWT)	Encrypted JSON Web Tokens with bearer verification middleware.
PDF Reporting	ReportLab Engine	Generates high-resolution weekly clinical receipts and dietary summaries.
HTTP Client	httpx (Async)	High-throughput asynchronous communication with external third-party APIs.

2.4 AI Engine & Multi-Key Failover Pool Architecture

NutriSense leverages Google's flagship Gemini models (gemini-2.5-flash and gemini-3.6-flash). To ensure 99.9% uptime and prevent service denial from Google API rate limits, a custom GeminiPool manager executes automatic round-robin load distribution and instant failover.

> **Resilient AI Pool Design Pattern**
> When an inference call triggers an HTTP 429 Quota Exceeded error, GeminiPool catches the exception, masks sensitive credentials in logs, instantly rotates the active index to the next verified key, and transparently completes inference in <500ms without user interruption.

2.5 External Integrations & Third-Party APIs

- OpenFoodFacts API: Global barcode lookup providing raw ingredient and nutritional metadata for packaged goods.
- OpenStreetMap / Overpass API: Geospatial query engine discovering certified emergency hospitals, clinics, and pharmacies within a 5km to 15km user radius.
- Aladhan Prayer API: Astronomical calculation engine providing exact daily Fajr (Sehri) and Maghrib (Iftar) timestamps based on GPS coordinates.

3. Software Engineering Diagrams & Specifications

Here are eight foundational architectural and behavioural diagrams specifying NutriSense's software design. Each specification outlines the actors, component interactions, lifecycle states, and data pathways.

3.1 Figure 1: High-Level System Architecture Diagram (Layered)

[Figure 1: Layered System Architecture Diagram]

(Architectural Diagram Placement Area)
Illustrates the 5 architectural tiers: Presentation Layer (Flutter UI), Controller & State Layer (Sync/Ramadan/Family Controllers), API Gateway (FastAPI REST), AI & External Services (Gemini Pool, OpenStreetMap, OpenFoodFacts), and Data Persistence (Supabase PostgreSQL + SQLite).

Architectural Walkthrough: The Presentation Layer communicates with the Controller Layer via reactive state notifications. The Controller Layer routes network requests through the ApiClient to FastAPI endpoints. FastAPI orchestrates Gemini AI inference and third-party APIs, persisting validated data to Supabase while the client maintains an offline-first SQLite mirror.

3.2 Figure 2: UML Use Case Diagram

[Figure 2: UML Use Case Diagram]

(Architectural Diagram Placement Area)
Illustrates primary actors (Primary User, Dependent Profile, AI Coach, Emergency Clinics) interacting with 9 core system use cases including AI Meal Logging, Voice Q&A, Barcode Allergen Check, Ramadan Fasting, Family Tracking, and Emergency Clinic Routing.

Use Case Specifications: The Primary User initiates core workflows (Meal Logging, Fasting Mode, Family Management). The AI Coach acts as an active actor providing proactive dietary recommendations and clinical symptom triage. External Emergency Services receive geolocation requests to provide direct turn-by-turn routing.

3.3 Figure 3: Entity-Relationship Diagram (Database Schema)

[Figure 3: Entity-Relationship Diagram (ERD / Database Schema)]

(Architectural Diagram Placement Area)
Illustrates relational tables in Supabase PostgreSQL: users, health_profiles, family_members, meal_logs, chat_history, water_logs, and habit_scores with 1:1, 1:N, and foreign key relationships.

The database schema is normalized to 3NF, enforcing referential integrity and user isolation via foreign key cascades:

Table Name	Key Columns & Types	Relational Constraints & Description
users	id (UUID PK), email, created_at	Root Supabase Auth entity representing registered accounts.
health_profiles	user_id (UUID FK), age, height, weight, goal, conditions[], allergies[]	1:1 with users. Stores clinical metrics, daily targets, and allergies.
family_members	id (UUID PK), primary_user_id (FK), name, relationship, age, conditions[]	1:N with users. Stores dependent sub-profiles for family tracking.
meal_logs	id (UUID PK), user_id (FK), family_member_id (FK), notes, calories, macros	Stores logged meals with macro breakdown and offline sync flags.

chat_history	id (UUID PK), user_id (FK), sender, message, metadata, timestamp	Maintains multi-turn conversations between user and AI Health Coach.
water_logs	id (UUID PK), user_id (FK), amount_ml, logged_at	Logs hydration entries for daily water target tracking.
habit_scores	id (UUID PK), user_id (FK), score, streak_days, last_calculated	Maintains 30-day algorithmic consistency scores and streaks.

3.4 Figure 4: Sequence Diagram – AI Meal Logging & Macro Estimation

[Figure 4: Sequence Diagram – AI Meal Logging Lifecycle]

(Architectural Diagram Placement Area)

Sequence: User -> Flutter UI (ManualLogScreen) -> FastAPI Backend (/meals/search-food) -> GeminiPool -> SQLite Cache -> Supabase Cloud DB.

Lifecycle Execution: (1) User inputs '1 plate Chicken Biryani with Raita'. (2) Flutter sends JSON payload to FastAPI `/meals/search-food`. (3) FastAPI prompts GeminiPool with USDA and South Asian culinary groundings. (4) Gemini returns structured JSON: 520 kcal, 28g protein, 65g carbs, 16g fat. (5) UI displays macro summary for 1-tap confirmation. (6) On confirmation, row is written to local SQLite and queued for background Supabase push.

3.5 Figure 5: Sequence Diagram – Gemini Multi-Key Failover Flow

[Figure 5: Sequence Diagram – Multi-Key Gemini Failover]

(Architectural Diagram Placement Area)

Sequence: Backend Client -> GeminiPool -> Key #1 (Throws 429 Quota Exceeded) -> Failover Catch -> Key #2 (Returns 200 OK) -> Response returned to Backend Client.

Failover Mechanism: When key quota is exhausted, the pool catches `google.genai.errors.ClientError (429)`, logs a warning with masked key identifiers, increments the active pointer, and re-executes inference with zero dropped packets.

3.6 Figure 6: UML Component Diagram

[Figure 6: UML Component Diagram]

(Architectural Diagram Placement Area)

Displays modular components across Client (UI, Controllers, SQLite Bridge, TTS), Application Backend (Auth Router, Meal Router, Coach Service, Report Engine), and Infrastructure Services.

Component Modularity: Ensures strict decoupling. The UI layer communicates solely through high-level ViewModels and Service singletons, allowing backend endpoints or AI models to be swapped without frontend refactoring.

3.7 Figure 7: State Machine / Activity Diagram – Offline Sync Workflow

[Figure 7: State Machine – Offline-First Sync Engine]

(Architectural Diagram Placement Area)

Workflow: App Online -> Write to SQLite & Supabase. App Offline -> Write to SQLite (is_synced=0) -> Network Reconnected -> SyncService background worker triggered -> Batch push to Supabase -> Update is_synced=1.

Data Synchronization Guarantee: The state machine guarantees zero data loss during network dropouts by treating local SQLite as the definitive client source of truth.

3.8 Figure 8: Deployment & Cloud Infrastructure Diagram

[Figure 8: Deployment & Cloud Topology Diagram]

(Architectural Diagram Placement Area)

Topology: Mobile Devices (Android APK / iOS) & Web Clients -> Cloudflare SSL Edge -> FastAPI Cloud Container (Docker on GCP / Render) -> Supabase Cloud (PostgreSQL + RLS + Storage) -> Google GenAI Cloud API.

Infrastructure Resilience: Employs TLS 1.3 encryption end-to-end, containerized stateless FastAPI application nodes, and managed Supabase PostgreSQL with automated daily backups.

4. Core Functional Module Specifications

4.1 Module 1: Natural Language AI Auto-Macro Estimator

The AI Auto-Macro Estimator eliminates the complexity of calorie counting. Users describe their meals in natural conversational language in English or Urdu without typing numbers.

- Bilingual Natural Language Parsing: Accurately understands colloquial meal names (e.g., '2 Aloo Parathas and 1 Chai', 'Daal Chawal with salad', 'Grilled Salmon with quinoa').
- Portion Intelligence: Automatically determines standard serving sizes when quantities are omitted, or scales macros dynamically based on specified quantities.
- Regional & Desi Cuisine Grounding: Built on an extensive South Asian nutritional knowledge matrix accounting for ghee, oil, and traditional cooking methods.

4.2 Module 2: High-Availability Multi-Key Gemini Failover Pool

To deliver enterprise SLA standards, the `GeminiPool` singleton implements thread-safe key management:

- Dynamic Round-Robin Rotation: Distributes API load across multiple Gemini API keys to balance quota consumption.
- Instant HTTP 429 Interception: Intercepts rate-limiting responses and switches keys in <50ms.
- Security Redaction: Automatically masks API key strings in server logs to prevent credential leakage.

4.3 Module 3: Multi-Member Family & Dependent Health Profiles

NutriSense allows a primary account holder to manage the nutritional wellness of their entire household:

- Dependent Profiles: Supports adding children, elderly parents, spouses, and siblings with custom age, gender, and calorie targets.
- Allergy & Medical Rules per Member: Enforces isolated safety guardrails (e.g., flagging peanut allergens for Child A while tracking low-sodium for Parent B).
- 1-Tap Dashboard Context Switching: Allows switching the primary dashboard to visualize meals and calorie progress for any family member instantly.

4.4 Module 4: Ramadan Fasting Intelligence & Hydration Engine

A dedicated operational mode tailoring the entire app experience for religious fasting observances:

- Solar Calculation Engine: Automatically fetches astronomical Sehri (Fajr) and Iftar (Maghrib) timings based on exact GPS coordinates.
- Live Fasting Countdown: Displays hours and minutes remaining until Iftar / Sehri with an animated circular progress indicator.
- Targeted Hydration Presets: Provides 1-tap logging chips tailored for non-fasting windows (+500ml Iftar, +250ml Taraweeh, +500ml Sehri).
- Dynamic Meal Categorization: Replaces standard Breakfast/Lunch slots with Sehri, Iftar, and Post-Taraweeh Snack.

4.5 Module 5: Real-Time AI Clinical Chat & Bilingual Voice Coach

An interactive AI dietitian available 24/7 for conversational guidance, recipe suggestions, and health habit evaluation:

- Rich Markdown Output: Renders bold key terms, bulleted dietary plans, and numbered instructions cleanly.
- Bilingual Text-To-Speech (TTS): Reads coaching responses aloud in natural English or Urdu via `flutter_tts`.
- Clinical Symptom Triaging: Detects red-flag medical emergencies (chest pain, acute hypoglycemia, severe breathlessness) and directs users to immediate medical care.

4.6 Module 6: Emergency Clinic & Hospital Geo-Finder

Provides life-saving access to healthcare facilities during dietary or medical emergencies:

- Overpass API Integration: Real-time geospatial queries scan certified clinics, hospitals, and pharmacies within a 5km to 15km radius.
- Facility Metadata: Displays distance in kilometers, 24/7 emergency readiness flags, and direct phone contact links.
- 1-Tap Route Navigation: Opens Google Maps or Apple Maps with pre-populated turn-by-turn emergency routing.

4.7 Module 7: Clinical Weekly Summary & PDF Receipt Generator

Aggregates 7-day health telemetry into professional clinical reports using ReportLab:

- Caloric & Macronutrient Balance: Visualizes average daily intake against prescribed medical targets.
- Consistency & Habit Scoring: Evaluates 30-day tracking consistency with streak metrics.
- Doctor-Ready PDF Receipt: Generates a downloadable clinical PDF summary suitable for sharing with physicians and dietitians.

5. Security, Compliance & Non-Functional Requirements

5.1 Authentication, Authorization & Session Security

- JWT Bearer Tokens: All client-server communications require cryptographically signed JSON Web Tokens issued by Supabase Auth.
- Secure Local Storage: Sensitive tokens and credentials are stored using hardware-backed platform keystores.
- Safe Account Termination: Comprehensive account deletion routines purge all relational database rows upon verified user confirmation.

5.2 Data Privacy & PostgreSQL Row-Level Security (RLS)

- Row-Level Security Policies: PostgreSQL RLS policies enforce that users can only SELECT, INSERT, UPDATE, or DELETE records where `user_id == auth.uid()`.
- Family Access Isolation: Dependent records and family meal logs are strictly restricted to the authenticated primary account holder.
- Zero AI Data Retention: Food scan queries and voice chats are processed ephemerally without using personal health records for public model training.

5.3 Performance, Scalability & Resource Optimization

- Sub-600ms AI Response Latency: Asynchronous FastAPI pipeline and optimized prompt tokens deliver instantaneous macro estimates.
- 60 FPS Smooth Rendering: Flutter UI utilizes const constructors, isolated ListenableBuilder rebuild scopes, and image caching.
- Low Power & Data Overhead: Background sync operations execute only upon network state change or batch queue triggers.

5.4 Offline Resilience & Idempotent Sync Integrity

- Zero Data Loss: Every meal, water, and habit log is committed immediately to SQLite before initiating network transmission.
- Idempotent Synchronization: Background sync operations utilize unique UUID keys to prevent duplicate records upon reconnecting.